Rapport P2I du 24_03_2026

semaine 7 **1**
Etape 1 1
Etape 2 1
Etape 3 2
Etape 4 3
Etape 5 3
Etape 6 4
Etape 7 4
Etape 8 4
Problème durant l'export 6
ajout d'un coef k pour contrôler le seuil de détection audio 6
ajout d'un coef pour contrôler le seuil de détection vidéo 7
Annexe **8**

semaine 7

Durant cette semaine, je vais m'intéresser à l'analyse audio, ce qui va me permettre d'avoir une analyse bien plus réaliste.

Etape 1

j'installe moviepy pour extraire l'audio : pip install moviepy

en haut de mon code analyze_video j'ajoute :

```
from moviepy import VideoFileClip
```

Etape 2

Mon premier objectif est de créer un score audio global

```
J'ajoute une fonction qui me permet d'ouvrire la vidéo, récupérer
l'audio, découper l'audio en petits morceaux synchronisés avec les
frames et calculer l'intensité moyenne de chaque morceau :

from moviepy import VideoFileClip
import numpy as np

def compute_audio_scores(video_path, fps):
    video = VideoFileClip(video_path)

    if video.audio is None:
        print("Pas de piste audio détectée")
        return np.array([])

    audio = video.audio.to_soundarray(fps=22050)

    # si stéréo, on moyenne les deux canaux
    if len(audio.shape) > 1:
        audio = np.mean(audio, axis=1)

    samples_per_frame = int(22050 / fps)
    audio_scores = []

    for i in range(0, len(audio), samples_per_frame):
        chunk = audio[i:i + samples_per_frame]

        if len(chunk) > 0:
            energy = np.mean(np.abs(chunk))
            audio_scores.append(energy)

    return np.array(audio_scores)

La fonction retourne une liste de score, plus la valeur est grande,
plus le moment a une intensité sonore forte.


## Etape 3


après motion_array = np.array(motion_scores)
j'ajoute :
audio_array = compute_audio_scores(video_path, fps)
```

Etape 4

Comme les valeurs n'ont pas la même échelle, il faut les ramener entre 0 et 1

J'ajoute une fonction pour normaliser:

```
def normalize_signal(signal):
    if len(signal) == 0:
        return signal

    min_val = np.min(signal)
    max_val = np.max(signal)

    if max_val - min_val == 0:
        return signal

    return (signal - min_val) / (max_val - min_val)
```

Puis dans analyze_video :

```
motion_norm = normalize_signal(motion_array)
audio_norm = normalize_signal(audio_array)
```

Etape 5

Je fusionne les deux :

```
combined_array = 0.7 * motion_norm + 0.3 * audio_norm
```

je met 70 pourcent pour l'image et 30 pourcent pour l'audio
Cela me paraît plutôt cohérent étant donné que l'analyse vidéo reste quand même plus importante.

Etape 6

```
Je remplace motion_array par combined_array ici :

smoothed_motion = moving_average(motion_array, window_size=7)
Devient :
smoothed_motion = moving_average(combined_array, window_size=7)
```

Etape 7

```
Ajout d'un seuil audio pour l'étude d'un graphique

if len(audio_array) > 0:
    audio_norm = normalize_signal(audio_array)
    audio_threshold = np.mean(audio_norm) + 0.7 * np.std(audio_norm)
    audio_peaks = np.where(audio_norm > audio_threshold)[0]
else:
    audio_norm = np.array([])
    audio_threshold = 0
    audio_peaks = []
```

Etape 8

```
Ajout du graphique audio juste en dessous du graphique vidéo: (Analyse
graphique)

if len(audio_norm) > 0:
    audio_plot_path = base_path / "outputs" / "audio_plot.png"

    plt.figure(figsize=(12, 6))
    plt.plot(audio_norm, label="Audio Score")
    plt.axhline(y=audio_threshold, color='r', linestyle='--',
label="Seuil audio")

    for peak in audio_peaks:
```

```
        plt.axvline(x=peak, color='orange', linestyle=':', alpha=0.6)

    plt.title("Analyse Audio")
    plt.xlabel("Frame Index")
    plt.ylabel("Audio Energy (normalisée)")
    plt.legend()

    plt.tight_layout()
    plt.savefig(audio_plot_path)
    plt.close()

    print(f"🔊 Graphe audio sauvegardé : {audio_plot_path}")
```

Le graphique montre la courbe d'énergie sonore, le seuil de détection et les pics.

Enfin J'ajoute un graphique des deux analyses combinés :

```
    combined_plot_path = base_path / "outputs" / "combined_plot.png"

plt.figure(figsize=(12, 6))
plt.plot(combined_array, label="Score fusionné")
plt.axhline(y=np.mean(combined_array) + 0.7 * np.std(combined_array),
            color='r', linestyle='--', label="Seuil fusionné")

plt.title("Analyse Fusionnée Audio + Vidéo")
plt.xlabel("Frame Index")
plt.ylabel("Score fusionné")
plt.legend()

plt.tight_layout()
plt.savefig(combined_plot_path)
plt.close()

print(f"Graphe fusionné sauvegardé : {combined_plot_path}")
```

Ce graphique permet de rendre compte des deux analyses (vidéo et audio)

Problème durant l'export

le son n'était pas présent lors de l'export du best-of, j'ai donc fais une modification dans la fonction export best of

l'utilisation de OpenCV permettait de générer le best-of vidéo, mais sans conserver la piste audio. Cette partie a donc été remplacée par un export utilisant moviepy, qui permet de découper et concaténer les segments tout en conservant le son original.

ajout d'un coef k pour contrôler le seuil de détection audio

Afin de rendre le système plus adaptable, le coefficient k utilisé dans le calcul du seuil audio a été rendu paramétrable. Ce paramètre permet d'ajuster la sensibilité de la détection audio selon le type de vidéo ou le niveau de sélectivité souhaité par l'utilisateur. Afin de rendre le système plus adaptable, le coefficient k utilisé dans le calcul du seuil audio a été rendu paramétrable. Ce paramètre permet d'ajuster la sensibilité de la détection audio selon le type de vidéo ou le niveau de sélectivité souhaité par l'utilisateur.

À quoi correspondent les choix :

sensible
k_audio = 0.4

Détecte plus de pics audio.
Utile si tu veux capter :

petits sons
réactions modérées
vidéos calmes

equilibree
k_audio = 0.7

Bon compromis.

```
C’est le réglage conseillé par défaut.

stricte
k_audio = 1.2

Ne garde que les sons vraiment marqués.
Utile pour éviter les faux positifs.
```

ajout d'un coef pour contrôler le seuil de détection vidéo

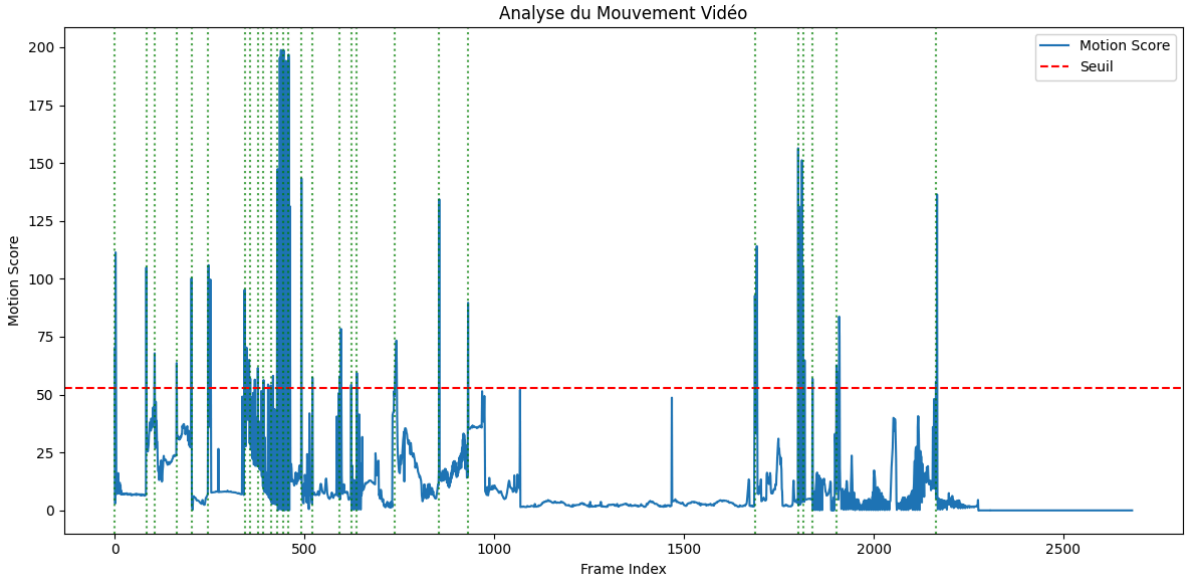
```
Les seuils de détection vidéo ont été rendus paramétrables afin de
permettre une adaptation du système au type de contenu analysé. Deux
paramètres distincts sont utilisés : l’un pour la détection des
coupures de scène et l’autre pour la détection des moments forts. Pour
faciliter l’usage, ces paramètres sont associés à trois niveaux de
sensibilité : sensible, équilibré et strict.

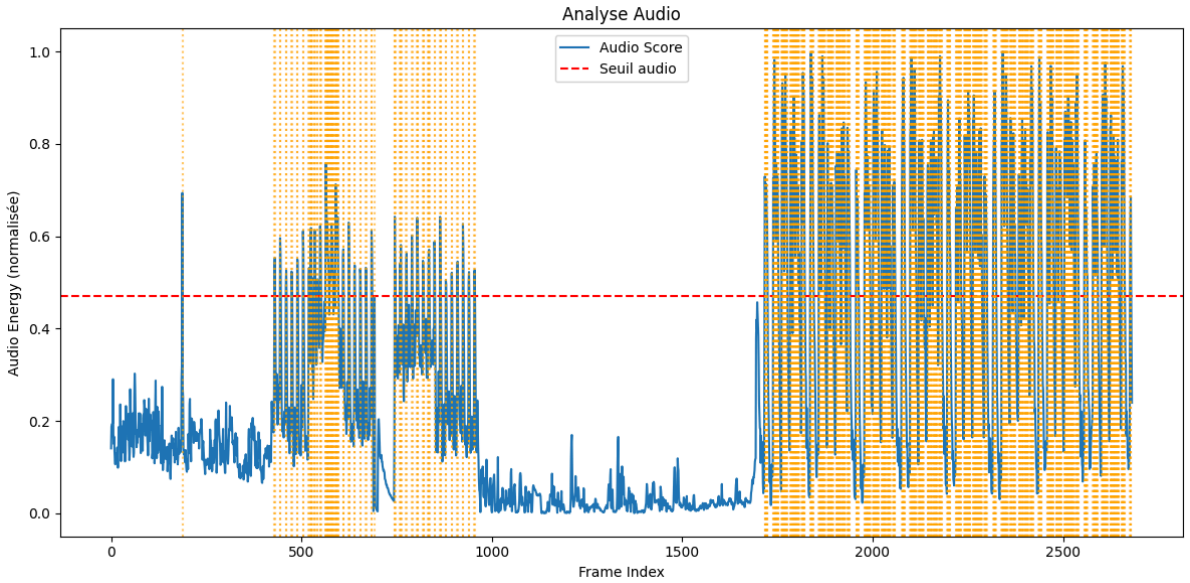
Sensible
k_video_cut = 1.5
k_video_highlight = 0.5
détecte plus de coupures
détecte plus de moments forts
plus sensible, mais plus de faux positifs

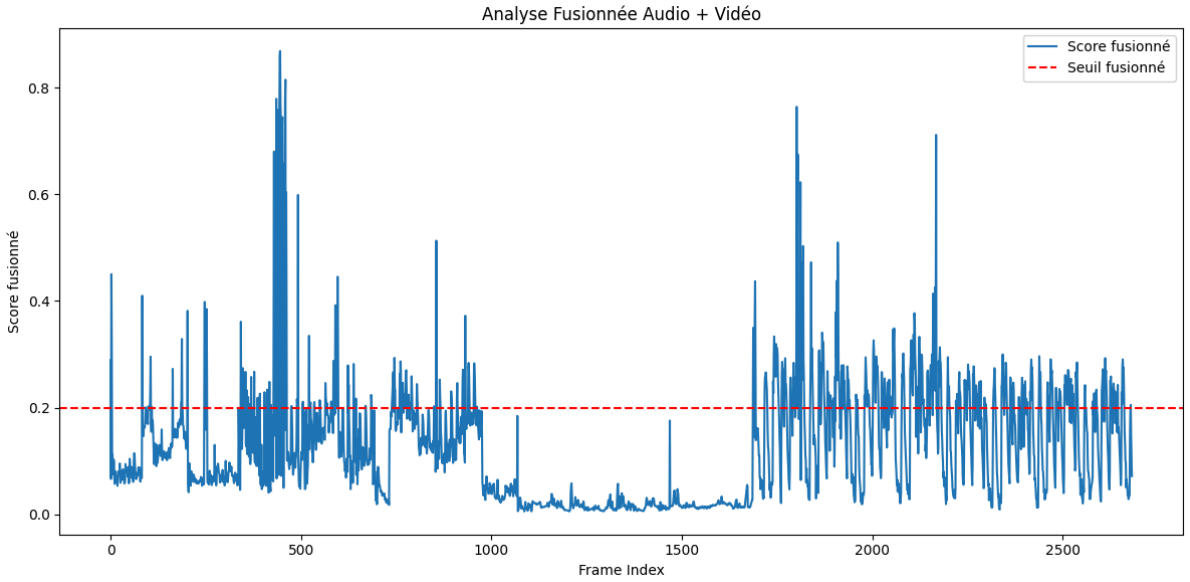
Équilibrée
k_video_cut = 2.0
k_video_highlight = 0.7
bon compromis
réglage conseillé par défaut

Stricte
k_video_cut = 2.5
k_video_highlight = 1.0
moins de coupures détectées
moins de moments forts
plus sélectif
```

Annexe


Analyse du Mouvement Vidéo
Motion Score
Seuil
Motion Score
Frame Index
0
25
50
75
100
125
150
175
200
0
500
1000
1500
2000
2500


Analyse Audio
Audio Score
Seuil audio
Audio Energy (normalisée)
Frame Index
0.0
0.2
0.4
0.6
0.8
1.0
0
500
1000
1500
2000
2500


Analyse Fusionnée Audio + Vidéo
Score fusionné
Seuil fusionné
Score fusionné
Frame Index
0.0
0.2
0.4
0.6
0.8
0
500
1000
1500
2000
2500